

An End-to-End AI Agent for Kaggle Competitions

Author: Wei-Hao Huang · **School:** National Chengchi University, Department of Economics
PI: Dr. Tso-Jung Yen, Institute of Statistical Science, Academia Sinica · Summer Internship

1 · Motivation

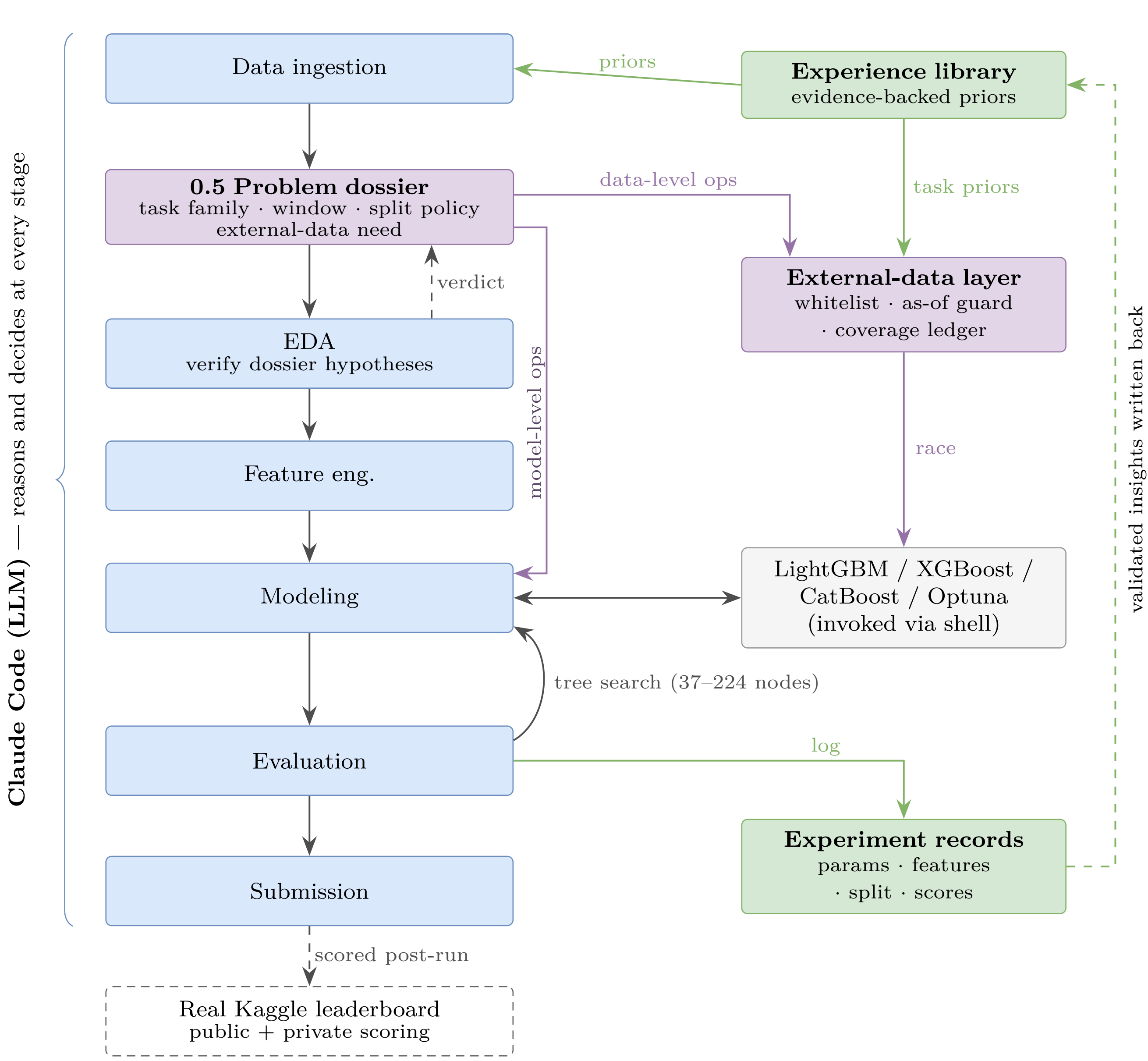
Challenges of Kaggle competitions

- The full expert loop: validation design, feature engineering, model selection, ensembling. Top entries are separated by a **very small margin**.
- **Data preparation** is essential but repetitive: every dataset needs the same cleaning, split design and drift checks before any model can be trusted.

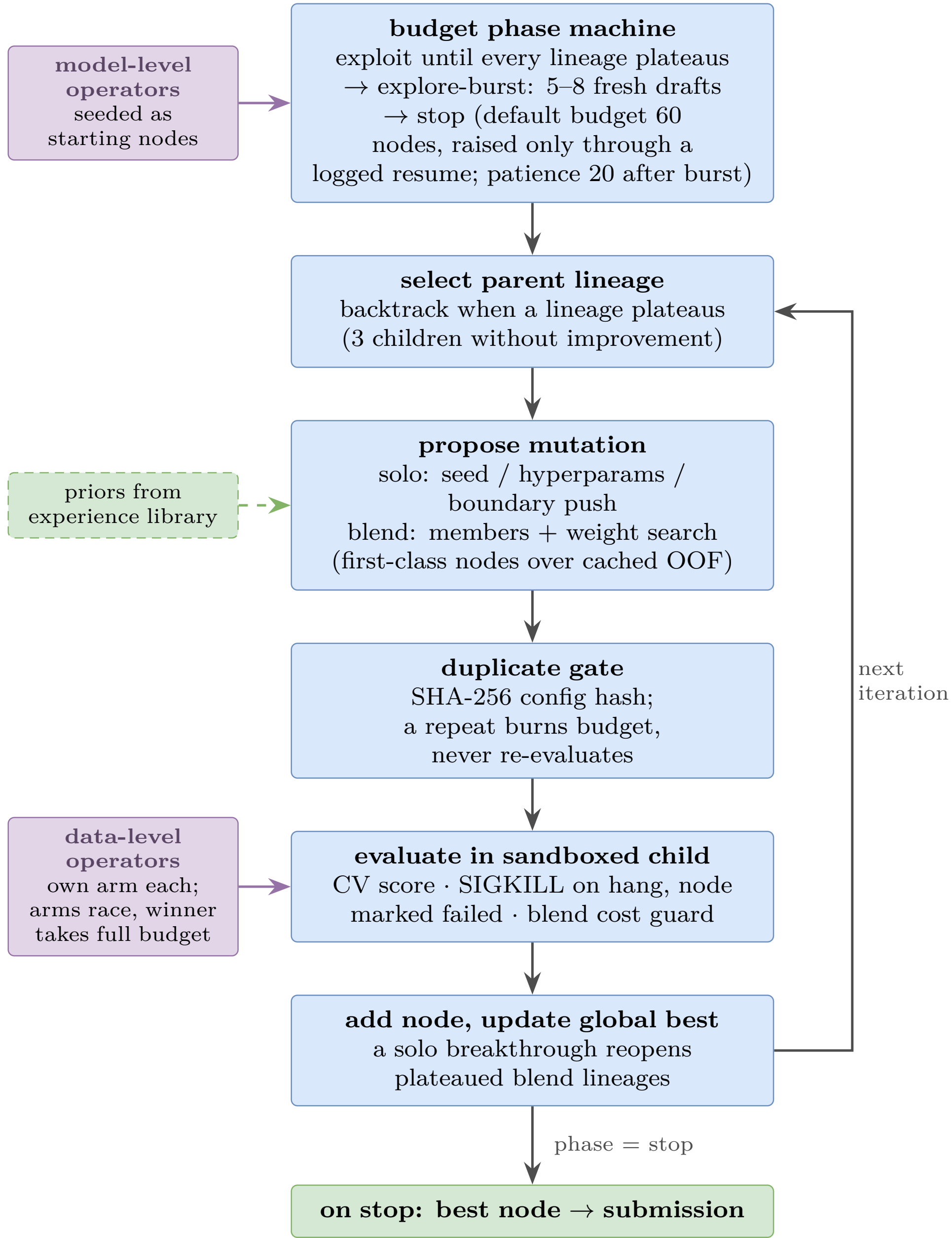
Why an AI agent

- A competition is a **scorable task**: every candidate returns a number, so it can be **optimized automatically** (propose, score, keep the better, repeat).
- Tireless, systematic search over candidate configurations, every experiment logged and reproducible.
- Records a per-dataset EDA pass and writes a per-competition report automatically, so every claim is grounded in an artifact.

2 · Workflow



The pipeline: the LLM reasons at every stage; the **problem dossier** classifies the task and injects external data *upstream*, before any modeling; submissions are scored post-run on the real leaderboard. Purple = injection layer · green = evidence loop · gray = Auto-ML tools; dashed = feedback or post-run.



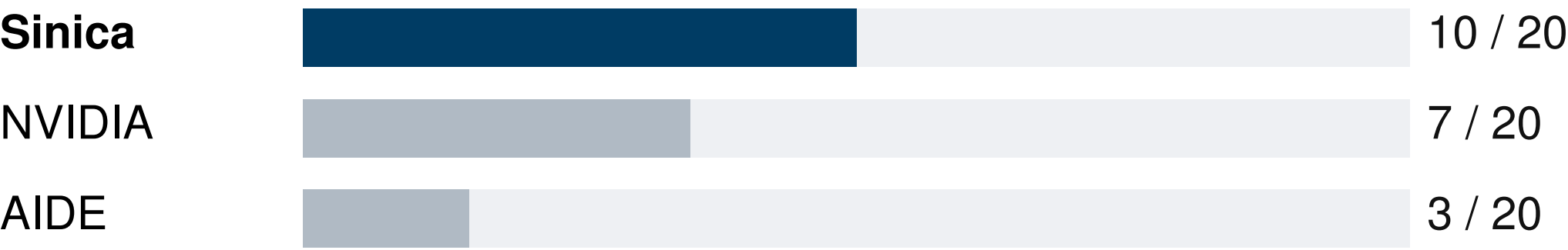
Tree-search loop: ensembles are first-class nodes; exploit until a lineage plateaus, then one forced exploration burst; the loop ends at the best-node submission.

3 · Results

Benchmark

All 20 competitions ran in sandboxes with a per-lane transcript audit, and all 20 are scored on the real leaderboard, as were the two reference agents. Each of the 60 head-to-head duels was judged against the score noise measured on Kaggle's own public/private split: 38 decided, 22 too close to call. **Sinica leads the standing: 16 wins / 8 losses (66.7% of its decided duels)** against 42.3% for each reference agent.

Best private score



Private-leaderboard scores

Competition	Metric	Sinica	NVIDIA	AIDE
s3e1	RMSE ↓	0.55275	0.55550	0.55552
s3e3	AUC ↑	0.87157	0.89537	0.86863
s3e5	QWK ↑	0.57223	0.56974	0.58138
s3e7	AUC ↑	0.90441	0.90458	0.90109
s3e9	RMSE ↓	12.3319	12.2291	12.3067
s3e11	RMSLE ↓	0.29368	0.29624	0.29445
s3e14	MAE ↓	333.186	330.716	332.316
s3e16	MAE ↓	1.33822	1.34001	1.34224
s3e19	SMAPE ↓	49.4088	48.2656	48.3413
s4e11	Acc. ↑	0.94107	0.94076	0.94056
s5e1	MAPE ↓	0.08401	0.14594	0.17917
s5e10	RMSE ↓	0.05576	0.05588	0.05579
s6e1	Error ↓	8.71516	8.73239	8.72191
s6e2	AUC ↑	0.95505	0.95508	0.95516
afsis	MCRMSE ↓	0.48917	0.69855	0.49861
cat-in-the-dat	AUC ↑	0.80243	0.77084	0.80217
conway	MAE ↓	0.10414	0.11189	0.11065
tps-aug-2022	AUC ↑	0.58930	0.58730	0.59106
tps-jan-2022	SMAPE ↓	6.29895	4.63651	10.6664
tps-sep-2022	SMAPE ↓	28.4961	5.48351	9.72095

Bold = best of the three.

Three mid-complexity competitions

Competition	Metric	Sinica	NVIDIA	AIDE
us-patent (NLP)	Pearson r ↑	0.86658	0.86160	0.65821
ventilator (time series)	MAE ↓	0.16525	0.40093	0.34591
SIIM-ISIC (imaging)	AUC ↑	0.93529	0.89822	0.88579

4 · Reflections

- **Put knowledge injection upstream.** At the blend stage it measured near-nothing and was abandoned; moved into problem identification it **improves scores significantly**.
- **No score until you submit.** A competition cannot be graded on the spot. While you iterate, local CV is all you see, so an error in problem-level judgment leaves CV looking healthy while the leaderboard says otherwise.
- **Freeze the architecture before you measure.** One results table can report only one agent version, so an architectural change invalidates every run before it. That is why such a change costs twice: a tail of bugs to fix, then a full re-run to earn the voided scores back.
- **Get isolation right.** If a lane can reach a leaderboard or another lane's work, every number in the table is worthless. It has to be enforced by the setup, not by telling the agent what to avoid.

5 · Weaknesses of AI-heavy research

- **Strong hands, no design judgment.** Implementation is fast and accurate, but the loop only solves the problem it is handed. It never asks whether a change is worth making, and nothing in it can tell whether an architecture holds until the runs come back, by which time the debt is already there.
- **The question gets lost in the detail.** Heavy AI use pulls a write-up toward the technical detail it can report exhaustively, and away from the one thing the reader came for.
- **AI writing habits survive review.** Redundant heading suffixes and invented terminology come back in every draft; keeping them out is manual work, not a setting.